

COURSE 8: DATA VISUALIZATION WITH PYTHON

MODULE 1– INTRODUCTION TO DATA VISUALIZATION TOOLS

1.1 INTRODUCTION TO DATA VISUALIZATION

1.1.1 What is Data Visualization?

Data Visualization is the **graphical representation of data and information** using charts, graphs, maps, dashboards, and plots.

It helps convert raw and complex data into visuals that are:

- Easy to understand
- Easy to analyze
- Visually appealing
- Useful for decision-making

1.1.2. Why is Data Visualization Important?

a. Simplifies Complex Data

Large datasets are difficult to understand in raw tabular form. Visualization makes them understandable.

Example:

Instead of reading thousands of rows of sales data:

- A line graph quickly shows sales trends.
- A bar chart compares product performance.

b. Identifies Patterns and Trends

Visualization helps detect:

- Growth trends
- Seasonal changes
- Relationships
- Outliers
- Correlations

c. Improves Communication

Visuals communicate insights faster than text or tables.

Stakeholders can:

- Understand findings quickly
- Make informed decisions
- Take action easily

d. Supports Better Decision-Making

Organizations use visualization to:

- Detect opportunities
- Identify risks
- Improve strategies

1.1.3 Real-World Uses of Data Visualization

Business

- Sales analysis
- Customer behavior
- Market trends
- Financial performance

Healthcare

- Patient monitoring
- Disease trends
- Treatment effectiveness

Education

- Student performance analysis
- Learning outcomes

Government

- Public statistics
- Census reports
- COVID dashboards

Entertainment

- Audience preferences
- Ratings analysis

1.1.4 Examples Mentioned in the Course

The New York Times

Used line charts and bar charts to show:

- COVID-19 cases
- Death rates
- Vaccination percentages

Airbnb

Used bar charts for:

- Smart pricing framework
- Lead time distribution

Spotify Pie

Used pie charts to visualize:

- Listening habits
- Genres
- Artists

Netflix

Used:

- Choropleth maps
- Dashboards
- Line charts
- Bar charts

1.1.5 Best Practices for Data Visualization

a. Choose the Correct Visualization Type

Data Type	Best Visualization
Trends over time	Line Chart
Category comparison	Bar Chart
Distribution	Histogram
Relationship between variables	Scatter Plot

b. Keep Visualizations Simple

Avoid:

- Too many colors
- Unnecessary graphics
- Overcrowded visuals

Simple visuals are easier to understand.

c. Label Everything Clearly

A good visualization should include:

- Title
- Axis labels
- Legends
- Units

d. Avoid Clutter

Only include relevant information.

Too many elements can confuse users.

e. Consider the Audience

Different audiences need different visual styles.

Example:

- Executives → dashboards and summaries
- Analysts → detailed plots and metrics

1.1.6 Good vs Bad Visualization

Bad Visualization

Problems:

- Too many variables
- Confusing labels
- Poor colors
- Hard to interpret

Good Visualization

Characteristics:

- Clear labels
- Proper scaling
- Consistent colors

- Easy interpretation

1.1.7 Darkhorse Analytics

A company specializing in:

- Data visualization
- Geospatial analysis

Main philosophy:

“Less is more.”

Simple visuals are:

- More effective
 - More attractive
 - More impactful
-

1.2 TYPES OF PLOTS

1.2.1 Line Plot

Definition

A line plot connects data points using straight lines.

Uses

- Trends over time
- Time-series analysis
- Continuous data

Examples

- Stock prices
- Website traffic
- Temperature changes

Advantages

- Easy trend identification
- Good for continuous data
- Highlights sudden changes

Important Note

Improper axis scaling can create misleading trends.

Example

If Y-axis does not start at 0:

- Small changes may appear huge.

1.2.2 Bar Plot

Definition

Uses rectangular bars to compare categories.

Types

- Vertical Bar Chart
- Horizontal Bar Chart

Uses

- Comparing groups
- Rankings
- Categorical data

Examples

- Sales by product
- Budget allocation
- Population comparison

Important Note

Improper scaling can exaggerate differences.

1.2.3 Scatter Plot

Definition

Displays values of two variables using points.

Uses

- Relationship analysis
- Correlation detection
- Cluster identification
- Outlier detection

Examples

- House price vs size
- Temperature vs electricity usage

Outliers

Outliers can:

- Distort interpretation
- Hide actual patterns

Proper handling is important.

1.2.4 Box Plot

Definition

Shows:

- Median
- Quartiles
- Spread
- Outliers

Also called:

Box-and-Whisker Plot

Uses

- Distribution comparison
- Skewness analysis
- Outlier detection

Examples

- Salary comparison across departments

1.2.5 Histogram

Definition

Represents data distribution using bins.

Uses

- Distribution analysis
- Shape analysis
- Skewness analysis
- Frequency analysis

Important Concept: Binning

Too Few Bins

- Oversimplifies data

Too Many Bins

- Creates noise

Optimal Bins

- Best representation
-

1.3 PLOT LIBRARIES IN PYTHON

1.3.1 Matplotlib

Definition

A general-purpose plotting library in Python.

Features

- Line plots
- Scatter plots
- Histograms
- Pie charts
- Box plots
- Heatmaps

Advantages

- Highly customizable
- Integrates with:
 - NumPy
 - Pandas
 - Seaborn
 - Plotly

1.3.2 Pandas Plotting

Features

- Built on Matplotlib
- Direct plotting from DataFrames

Useful For

- Exploratory Data Analysis (EDA)

1.3.3 Seaborn

Definition

Statistical visualization library built on Matplotlib.

Features

- Attractive default themes
- Heatmaps
- Violin plots
- Regression plots
- Grid layouts

1.3.4 Folium

Definition

Library for geospatial visualization.

Features

- Interactive maps
- Choropleth maps
- Heat maps
- Marker maps

1.3.5 Plotly

Definition

Interactive visualization library.

Features

- Interactive charts
- Dashboards
- 3D plots
- Web-based visualization

1.3.6 PyWaffle

Purpose

Creates waffle charts for proportional data visualization.

1.4 INTRODUCTION TO MATPLOTLIB

1.4.1 What is Matplotlib?

Matplotlib is one of the most widely used visualization libraries in Python.

Created by:

John Hunter

1.4.2 Why Was Matplotlib Created?

John Hunter wanted to replace expensive proprietary EEG/ECoG visualization software with an open-source alternative.

Initially developed for:

- EEG visualization
- ECoG visualization

1.4.3 Matplotlib Architecture

1. Backend Layer

Handles:

- Rendering
- User interaction
- Figure canvas

Components

- FigureCanvas
- Renderer
- Event handler

2. Artist Layer

Everything visible on the plot is an artist.

Primitive Artists

- Line
- Rectangle
- Circle
- Text

Composite Artists

- Axis
- Tick
- Figure

3. Scripting Layer (Pyplot)

Most commonly used layer.

Provides:

- Simple plotting interface
- Easy figure generation

Imported as:

```
import matplotlib.pyplot as plt
```

1.4.4 Anatomy of a Plot

Main components:

- Figure
- Axes
- Axis
- Title
- Labels
- Legends

- Grid
 - Data points
 - Annotations
-

1.5 JUPYTER NOTEBOOK & MATPLOTLIB

1.5.1 Jupyter Notebook

Open-source environment for:

- Live code
- Visualizations
- Documentation

1.5.2 Magic Function

```
%matplotlib inline
```

Used to display plots inside notebook cells.

1.5.3 Basic Plotting with Matplotlib

Import Libraries

```
import matplotlib.pyplot as plt
```

Simple Plot Example

```
plt.plot(5,5,'o')  
plt.show()
```

Adding Labels and Title

```
plt.title("Sample Plot")  
plt.xlabel("X-axis")  
plt.ylabel("Y-axis")
```

1.5.4 Plotting with Pandas

Line Plot

```
df.plot(kind='line')
```

Histogram

```
df.plot(kind='hist')
```

1.6 DATASET USED IN THE COURSE

1.6.1 Dataset

UN Immigration Data to Canada

Contains:

- Country
- Region
- Development status
- Immigration data (1980–2013)

1.6.2 Importing Dataset

```
import pandas as pd
```

```
df = pd.read_excel('Canada.xlsx')
```

1.6.3 Data Processing Steps

1. Set Country as Index

```
df.set_index('Country', inplace=True)
```

2. Create Total Column

```
df['Total'] = df.sum(axis=1)
```

1.7 LINE PLOTS

1.7.1 Definition

A series of data points connected by straight lines.

1.7.2 When to Use Line Plots

Best For:

- Trends over time
- Continuous data
- Time-series analysis

1.7.3 Haiti Immigration Example

A line plot showed a major immigration spike from Haiti in 2010.

Reason:

- Haiti earthquake in 2010

This demonstrates how visualization helps identify:

- Trends
- Events
- Anomalies

1.7.4 Creating a Line Plot

```
haiti = df.loc['Haiti', years]

haiti.plot(kind='line')

plt.title('Immigration from Haiti')
plt.xlabel('Years')
plt.ylabel('Number of Immigrants')

plt.show()
```

KEY TAKEAWAYS FROM MODULE 1

You Learned:

- Importance of data visualization
- Different plot types
- Best visualization practices
- Popular Python visualization libraries
- Matplotlib architecture
- Jupyter notebook integration
- Data preprocessing with Pandas
- Creating line plots using Matplotlib
- Working with immigration dataset
- Basics of exploratory visualization

[Hands-On Lab: Exploring And Pre-Processing A Dataset Using Pandas](#)

[Hands-on Lab: Introduction to Matplotlib and Line Plots](#)

MODULE 2: BASIC AND SPECIALIZED VISUALIZATION TOOLS

BASIC VISUALIZATION TOOLS

2.1 AREA PLOT

2.1.1 What is an Area Plot?

An **Area Plot** (or Area Chart) is similar to a line plot, but the area under the line is filled with color. It is used to show:

- Magnitude of change over time
- Cumulative totals
- Comparison between multiple variables

It is especially useful for:

- Population growth
- Stock market trends
- Immigration statistics
- Resource distribution

2.1.2 How to Create an Area Plot using Matplotlib

Step 1: Import Libraries

```
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
```

Step 2: Prepare the Data

```
# Set country names as index
df_can.set_index('Country', inplace=True)

# Create total column
df_can['Total'] = df_can.sum(axis=1)

# Sort values by total immigration
df_can.sort_values(['Total'], ascending=False, inplace=True)
```

Step 3: Select Top 5 Countries

```
years = list(map(str, range(1980, 2014)))
df_top5 = df_can.head()[years].transpose()
df_top5.head()
```

Why transpose (transpose() or .T)?

Because:

- Matplotlib uses DataFrame index on X-axis
- We want years on X-axis
- Countries should become columns

Step 4: Create Area Plot

```
df_top5.plot(kind='area',  
             figsize=(12,6))  
plt.title('Immigration Trend of Top 5 Countries')  
plt.xlabel('Years')  
plt.ylabel('Number of Immigrants')  
plt.show()
```

2.1.3 Important Points about Area Plots

- `kind='area'` creates an area chart
 - Best for cumulative data
 - Useful for comparing multiple trends together
 - Filled colors improve visualization
-

2.2 HISTOGRAM

2.2.1 What is a Histogram?

A **Histogram** shows the frequency distribution of numerical data.

It:

- Divides data into intervals called bins
- Counts how many values fall into each bin

Example Uses

- Exam scores distribution
- Salary ranges
- Immigration distribution
- Age distribution

2.2.2 Creating a Histogram using Matplotlib

Basic Histogram

```
df_can[2013].plot(kind='hist')
plt.title('Histogram of Immigration from 195 Countries in 2013')
plt.xlabel('Number of Immigrants')
plt.ylabel('Number of Countries')
plt.show()
```

Improving Histogram using NumPy Bins

Step 1: Import NumPy

```
import numpy as np
```

Step 2: Generate Bins

```
count, bin_edges = np.histogram(df_can[2013])
```

Step 3: Plot Histogram

```
df_can[2013].plot(kind='hist',
                  figsize=(8,5),
                  xticks=bin_edges)
plt.title('Histogram of Immigration in 2013')
plt.xlabel('Number of Immigrants')
plt.ylabel('Number of Countries')
plt.show()
```

2.2.3 Important Points about Histograms

- `kind='hist'` creates histogram
- Shows distribution of numeric data
- bins divide data into intervals
- NumPy helps align bin edges properly

2.3 Bar Charts

2.3.1 What is a Bar Chart?

A **Bar Chart** compares values across categories.

- Height (or length) of bar represents value
- Useful for comparing yearly or categorical data

Example Uses

- Sales comparison
- Immigration comparison
- Student marks
- Population comparison

2.3.2 Creating a Vertical Bar Chart

Step 1: Select Iceland Data

```
years = list(map(str, range(1980, 2014)))  
df_iceland = df_can.loc['Iceland', years]
```

Step 2: Plot Bar Chart

```
df_iceland.plot(kind='bar',  
                figsize=(10,6))  
plt.title('Iceland Immigration to Canada')  
plt.xlabel('Year')  
plt.ylabel('Number of Immigrants')  
plt.show()
```

2.3.3 Horizontal Bar Chart

```
df_iceland.plot(kind='barh',  
                color='green')  
plt.title('Horizontal Bar Chart')  
plt.xlabel('Number of Immigrants')  
plt.ylabel('Year')  
plt.show()
```

2.3.4 Customizing Bar Colors

```
colors = ['red' if (x == 1981 or x == 1990)
          else 'blue' for x in range(1980,2014)]
df_iceland.plot(kind='bar',
                color=colors,
                edgecolor='black')
plt.title('Customized Bar Chart')
plt.show()
```

QUICK REVISION TABLE

Plot Type	Purpose	Plot Parameter
Line Plot	Trends over time	kind='line'
Area Plot	Cumulative trends	kind='area'
Histogram	Frequency distribution	kind='hist'
Bar Chart	Category comparison	kind='bar'
Horizontal Bar	Horizontal comparison	kind='barh'

KEY TAKEAWAYS

Area Plot

- Filled version of line plot
- Best for cumulative values
- Use: `plot(kind='area')`

Histogram

- Shows frequency distribution
- Uses bins
- Use: `plot(kind='hist')`

Bar Chart

- Compares categories
- Vertical or horizontal bars
- Use: `plot(kind='bar')`
`plot(kind='barh')`

[Hands-on Lab: Area Plots, Histograms, and Bar Charts](#)

SPECIALIZED VISUALIZATION TOOLS

2.4 PIE CHART

2.4.1 What Is a Pie Chart?

A **pie chart** is a circular graph divided into slices where each slice represents a proportion or percentage of the whole.

Uses

- Showing percentage distribution
- Comparing proportions among categories
- Visualizing contribution of parts to a whole

Example

Continent-wise immigration to Canada:

- Asia → largest share
- Europe → smaller share
- Africa → moderate share

2.4.2 Create Pie Chart using Matplotlib

```
import matplotlib.pyplot as plt
sizes = [40, 25, 20, 15]
labels = ['Asia', 'Europe', 'Africa', 'America']
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title('Immigration by Continent')
plt.show()
```

Important Parameters

- labels → names of slices
 - autopct → display percentages
 - explode → separates slices for emphasis
-

2.5 BOX PLOT

2.5.1 What is a Box Plot?

A **box plot** statistically represents data distribution using five-number summary:

1. Minimum
2. First Quartile (Q1)
3. Median
4. Third Quartile (Q3)
5. Maximum

It also displays **outliers**.

Uses

- Understanding distribution
- Detecting outliers
- Comparing datasets

2.5.2 Create Box Plot using Matplotlib

```
import matplotlib.pyplot as plt
data = [10, 20, 30, 40, 50, 60, 100]
plt.boxplot(data)
plt.title('Box Plot Example')
plt.show()
```

Interpretation

- Middle line → median
 - Box → interquartile range
 - Dots outside → outliers
-

2.6 SCATTER PLOT

2.6.1 What is a Scatter Plot?

A **scatter plot** displays values for two variables using points on Cartesian coordinates.

Uses

- Finding relationships between variables
- Detecting trends/correlation
- Identifying clusters or anomalies

Example

Education vs Income:

- More education → higher income trend

2.6.2 Create Scatter Plot using Matplotlib

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 35, 45]
plt.scatter(x, y)
plt.title('Scatter Plot Example')
plt.xlabel('Education')
plt.ylabel('Income')
plt.show()
```

Important Parameters

- color
 - marker
 - s → marker size
-

2.7 TREEMAPS

2.7.1 What is a Treemap?

A **treemap** visualizes hierarchical data using nested rectangles.

- Rectangle size → value magnitude
- Nested rectangles → hierarchy

Uses

- Sales analysis
- Portfolio visualization
- File system storage analysis
- Website traffic analysis

2.7.2 Create Treemap using Plotly

```
import plotly.express as px
import pandas as pd
data = {
    'Category': ['Electronics', 'Electronics', 'Furniture'],
    'Subcategory': ['Laptops', 'Phones', 'Chairs'],
    'Sales': [120000, 80000, 50000]
}
df = pd.DataFrame(data)
fig = px.treemap(
    df,
    path=['Category', 'Subcategory'],
    values='Sales',
    title='Sales Treemap'
)
fig.show()
```

2.8 PIVOT CHARTS

2.8.1 What is a Pivot Chart?

A **pivot chart** summarizes and visualizes data from a pivot table dynamically.

Uses

- Business intelligence
- Financial analysis
- Sales trend analysis
- Marketing dashboards

2.8.2 Create Pivot Table and Pivot Chart

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    'Quarter': ['Q1', 'Q1', 'Q2', 'Q2'],
    'Category': ['Laptops', 'Desktops', 'Laptops', 'Desktops'],
    'Sales': [5000, 3000, 7000, 4000]
}
df = pd.DataFrame(data)
pivot = df.pivot_table(
    values='Sales',
    index='Quarter',
    columns='Category',
    aggfunc='sum'
)
pivot.plot(kind='bar')
plt.title('Sales Pivot Chart')
plt.xlabel('Quarter')
plt.ylabel('Sales')
plt.show()
```

[Hands-on Lab: Pie Charts, Box Plots, Scatter Plots, and Bubble Plots](#)

2.9 PLOTTING DIRECTLY WITH MATPLOTLIB

2.9.1 Plotting Directly with Matplotlib

Import Libraries

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

Line Plot

```
plt.plot(x, y)
```

Area Plot

```
plt.fill_between(x, y)
```

Histogram

```
plt.hist(data, bins=10)
```

Bar Chart

```
plt.bar(x, height)
```

Pie Chart

```
plt.pie(sizes, labels=labels)
```

Box Plot

```
plt.boxplot(data)
```

Scatter Plot

```
plt.scatter(x, y)
```

Subplots

```
fig, axes = plt.subplots(2, 2)
```

2.9.2 Plot Customization

Add Title and Labels

```
plt.title('My Plot')  
plt.xlabel('X-axis')  
plt.ylabel('Y-axis')
```

Add Grid

```
plt.grid(True)
```

Add Legend

```
plt.legend()
```

Axis Limits

```
plt.xlim(0, 10)  
plt.ylim(0, 100)
```

2.9.3 Data Storytelling vs Data Visualization

Data Storytelling	Data Visualization
Creates narrative around data	Creates graphical representation
Focuses on communication	Focuses on visual understanding
Combines context + visuals	Focuses on charts and plots

[Hands-on Lab: Plotting Directly with Matplotlib](#)

MODULE 3: ADVANCED VISUALIZATIONS AND GEOSPATIAL DATA

ADVANCED VISUALIZATION TOOLS

3.1 WAFFLE CHARTS

3.1.1 What is a Waffle Chart?

A **waffle chart** is a visualization technique that represents categorical data using a grid of small squares (tiles).

- Each square represents a value or percentage.
- Color and number of squares show proportions of categories.

It looks similar to a tiled grid.

3.1.2 Features of Waffle Charts

- Easy to understand
- Visually attractive
- Good for non-technical audiences
- Highlights proportions clearly

3.1.3 Use Cases of Waffle Charts

1. Market Share Analysis

Shows percentage share of companies/products.

2. Demographic Representation

Displays population categories such as:

- Age groups
- Gender
- Ethnicities

3. Project Progress Tracking

Visualizes completed vs remaining tasks.

4. Budget Allocation

Shows distribution of budget among departments.

5. Survey Results

Displays responses to multiple-choice questions.

6. Election Results

Represents vote distribution among parties.

7. Product Sales Analysis

Shows sales distribution across categories or regions.

3.1.4 Create Waffle Chart using PyWaffle

Install Library

```
pip install pywaffle
```

Example

```
from pywaffle import Waffle
import matplotlib.pyplot as plt
data = {
    'Asia': 50,
    'Europe': 20,
    'Africa': 15,
    'America': 15
}
fig = plt.figure(
    FigureClass=Waffle,
    rows=10,
    values=data,
    title={'label': 'Immigration Distribution'}
)
plt.show()
```

3.2 WORD CLOUD

3.2.1 What is a Word Cloud?

A **word cloud** is a text visualization technique where:

- Frequently appearing words appear larger and bolder.
- Less frequent words appear smaller.

It visually highlights important keywords.

3.2.2 Features of Word Cloud

- Summarizes text quickly
- Identifies important keywords
- Visually engaging
- Easy to interpret

3.2.3 Use Cases of Word Cloud

1. Social Media Analysis

Find trending hashtags or topics.

2. Customer Feedback Analysis

Identify common customer complaints or praises.

3. Content Analysis

Analyze blogs, articles, and research papers.

4. Market Research

Extract insights from surveys/interviews.

5. Resume & Job Description Analysis

Highlight important skills and keywords.

3.2.4 Create Word Cloud in Python

Install Library

```
pip install wordcloud
```

Example

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
text = "Python Data Visualization Python Matplotlib Seaborn Data"
wordcloud = WordCloud(
    background_color='white'
).generate(text)
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```

3.3 SEABORN AND REGRESSION PLOTS

3.3.1 What is Seaborn?

Definition

Seaborn is a statistical data visualization library built on top of Matplotlib.

It provides:

- Better visual appearance
- Built-in themes and color palettes
- Easier syntax
- Statistical plotting capabilities

Advantages of Seaborn

Feature	Benefit
Built on Matplotlib	Powerful and flexible
Attractive themes	Better visuals
Less code	Faster plotting
Statistical plots	Easy analysis
Works with NumPy/Pandas	Good integration

Specialized Plots in Seaborn

- Regression plots
- Distribution plots
- Categorical plots
- Heatmaps

3.3.2 Regression Plot (Regplot)

What is a Regression Plot?

A regression plot:

- Displays scatter points
- Adds a regression line
- Shows trend/correlation between variables

Create Regression Plot using Seaborn

Example

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
df = pd.DataFrame({
    'Year': [2000, 2001, 2002, 2003, 2004],
    'Immigration': [1000, 1500, 2000, 2500, 3000]})
sns.regplot(
    data=df,
    x='Year',
    y='Immigration'
)
plt.title('Regression Plot')
plt.show()
```

Customization in Regplot

Change Color

```
sns.regplot(
    data=df,
    x='Year',
    y='Immigration',
    color='green'
)
```

Change Marker

```
sns.regplot(  
    data=df,  
    x='Year',  
    y='Immigration',  
    marker='+'  
)
```

3.3.3 Count Plot

What is a Count Plot?

A count plot shows the frequency of categories.

Example

```
sns.countplot(  
    x='Continent',  
    data=df_canada  
)
```

3.3.4 Bar Plot in Seaborn

Example

```
sns.barplot(  
    x='Continent',  
    y='Total',  
    data=df_canada  
)
```

- Automatically calculates aggregated values
 - Displays confidence intervals
-

IMPORTANT SUMMARY

Visualization Tool	Purpose
Waffle Chart	Show proportions using square grids
Word Cloud	Highlight important words in text
Seaborn	Statistical visualization
Regression Plot	Show relationship between variables
Count Plot	Show category frequencies
Bar Plot	Compare categorical values

KEY POINTS

- Waffle charts represent categorical data using square tiles.
- Word clouds visualize important words based on frequency.
- Seaborn is built on top of Matplotlib.
- Seaborn simplifies statistical plotting.
- Regression plots can be created with one line of code using `sns.regplot()`.
- Seaborn provides attractive themes and color palettes.
- Waffle charts and word clouds are useful for storytelling and presentation.

[Hands-on Lab: Waffle Charts, Word Clouds, and Regression Plots](#)

3.4 INTRODUCTION TO FOLIUM

3.4.1 What is Folium?

Folium is a Python library used for visualizing **geospatial data**.

It creates:

- Interactive maps
- Marker maps
- Choropleth maps
- Clustered maps

3.4.2 Features of Folium

- Interactive zooming
- Multiple map styles
- Add markers and popups
- Geospatial visualization
- Choropleth maps

3.4.3 Creating a Basic Map

Example

```
import folium
world_map = folium.Map()
world_map
```

3.4.4 Creating a Map Centered on Canada

```
canada_map = folium.Map(
    location=[56.130, -106.35],
    zoom_start=4
)
canada_map
```

3.4.5 Different Map Styles

Stamen Toner

```
folium.Map(  
    location=[56.130, -106.35],  
    zoom_start=4,  
    tiles='Stamen Toner'  
)
```

Stamen Terrain

```
folium.Map(  
    location=[56.130, -106.35],  
    zoom_start=4,  
    tiles='Stamen Terrain'  
)
```

3.5 MAPS WITH MARKERS

3.5.1 What are Markers?

Markers indicate:

- Specific locations
- Points of interest
- Important geographic positions

3.5.2 Adding a Marker

```
import folium  
canada_map = folium.Map(  
    location=[56.130, -106.35],  
    zoom_start=4)  
folium.Marker(  
    location=[51.2538, -85.3232],  
    popup='Ontario'  
)add_to(canada_map)  
canada_map
```

3.5.3 Circle Marker Example

```
folium.CircleMarker(  
    location=[51.2538, -85.3232],  
    radius=10,  
    color='red',  
    fill=True,  
    fill_color='red'  
).add_to(canada_map)
```

3.5.4 Multiple Markers

```
locations = [  
    [43.6532, -79.3832],  
    [45.4215, -75.6972],  
    [49.2827, -123.1207]  
]  
for loc in locations:  
    folium.Marker(location=loc).add_to(canada_map)
```

3.5.5 Marker Clusters

Useful when there are many markers.

```
from folium.plugins import MarkerCluster  
marker_cluster = MarkerCluster().add_to(canada_map)  
for loc in locations:  
    folium.Marker(location=loc).add_to(marker_cluster)
```

3.6 CHOROPLETH MAPS

3.6.1 What is a Choropleth Map?

A **Choropleth Map** is a thematic map where regions are shaded according to statistical values.

- Darker color = higher value
- Lighter color = lower value

3.6.2 Uses of Choropleth Maps

- Population density
- Income distribution
- Crime rates
- Immigration statistics
- Election analysis

3.6.3 Requirements for Choropleth Maps

Folium requires:

1. Geographic data (GeoJSON)
2. Statistical dataset (DataFrame)

3.6.4 Creating a Choropleth Map

```
folium.Choropleth(  
    geo_data='world_geo.json',  
    data=df_canada,  
    columns=['Country', 'Total'],  
    key_on='feature.properties.name',  
    fill_color='YlOrRd',  
    fill_opacity=0.7,  
    line_opacity=0.2,  
    legend_name='Immigration to Canada'  
)add_to(world_map)
```

IMPORTANT FOLIUM PARAMETERS

Parameter	Purpose
location	Center coordinates
zoom_start	Initial zoom level
tiles	Map style
popup	Label on click
geo_data	GeoJSON file
fill_color	Choropleth color scheme

KEY TAKEAWAYS

Waffle Charts

- Represent categorical proportions using tiles.
- Created using PyWaffle.

Word Cloud

- Highlights frequent words visually.
- Created using WordCloud library.

Seaborn

- Statistical visualization library built on Matplotlib.
- Easier and more attractive plots.

Folium

- Creates interactive geospatial maps.
- Supports markers, clusters, and choropleth maps.

Choropleth Maps: Use color intensity to represent data values geographically.

[Hands-on Lab: Creating Maps and Visualizing Geospatial Data](#)

MODULE 4: CREATING DASHBOARDS WITH PLOTLY AND DASH

4.1 DASHBOARDING OVERVIEW

4.1.1 What is a Dashboard?

A **dashboard** is an interactive visual interface that displays:

- Important business metrics
- Charts and graphs
- Real-time insights
- KPIs (Key Performance Indicators)

Dashboards help organizations:

- Monitor performance
- Analyze trends
- Make data-driven decisions

4.1.2 Benefits of Dashboards

1. Easy Understanding

Dashboards present data visually using:

- Charts
- Maps
- Tables
- Graphs

This makes data easier to understand.

2. Real-Time Monitoring

Dashboards can display:

- Live data
- Real-time updates
- Current business status

3. Better Decision Making

Stakeholders can:

- Identify issues quickly
- Compare performance
- Track goals

4. Interactive Exploration

Users can:

- Hover over charts
- Drill down into details
- Filter information dynamically

4.1.3 Example: Flight Performance Dashboard

A dashboard may show:

- Top airline carriers
- Monthly flight counts
- Passenger statistics
- Distance group analysis

Compared to static tables, dashboards:

- Are more attractive
- Are easier to analyze
- Provide interactive exploration

4.1.4 Dashboarding Tools in Python

1. Dash

Built on:

- Flask
- Plotly.js

- React.js

Used for:

- Interactive analytics applications
- Custom dashboards

2. Streamlit

Converts Python scripts into web apps easily.

Features:

- Simple syntax
- Fast deployment
- Interactive widgets

3. Panel

Works with:

- Matplotlib
- Bokeh
- HoloViews

Useful for:

- Jupyter dashboards
- Standalone apps

4. Voila

Converts Jupyter notebooks into web applications.

5. Bokeh

Interactive plotting and dashboard library.

6. Flask

Python web framework used to build custom web dashboards.

4.1.5 Key Point

Data Storytelling vs Data Visualization

Data Visualization	Data Storytelling
Charts and graphs	Narrative around insights
Shows data visually	Explains meaning of data
Focuses on representation	Focuses on communication

4.2 INTRODUCTION TO PLOTLY

4.2.1 What is Plotly?

Plotly is an interactive open-source visualization library.

Supports:

- 40+ chart types
- Interactive plots
- Web-based visualizations

Available in:

- Python
- JavaScript

4.2.2 Features of Plotly

- Interactive charts
- Zooming and hovering
- Animation support
- Dashboard integration
- HTML export
- Beautiful visual appearance

4.2.3 Plotly Submodules

1. Plotly Graph Objects (`graph_objects`)

Low-level interface for:

- Figures
- Layouts
- Traces

Provides detailed customization.

Imported as:

```
import plotly.graph_objects as go
```

2. Plotly Express (`plotly.express`)

High-level wrapper around graph objects.

Advantages:

- Less code
- Easier syntax
- Faster plotting

Imported as:

```
import plotly.express as px
```

4.2.4 Plotly Graph Objects Example

Line Plot

```
import plotly.graph_objects as go
import numpy as np
x = np.arange(12)
y = np.random.randint(50, 500, size=12)
fig = go.Figure(data=go.Scatter(x=x, y=y))
fig.update_layout(
    title='Simple Line Plot',
    xaxis_title='Month',
    yaxis_title='Sales')
fig.show()
```

4.2.5 Plotly Express Example

Same Line Plot with Less Code

```
import plotly.express as px
import numpy as np
x = np.arange(12)
y = np.random.randint(50, 500, size=12)
fig = px.line(
    x=x,
    y=y,
    title='Simple Line Plot',
    labels={'x':'Month', 'y':'Sales'}
)
fig.show()
```

Advantages of Plotly Express

- Simple syntax
 - Fast development
 - Automatically interactive
 - Easy customization
-

4.3 Plotly Visualization Types

4.3.1 Scatter Plot

Purpose

Shows relationship between two variables.

Example

```
fig = px.scatter(df, x='Height', y='Weight')
```

4.3.2 Line Plot

Purpose

Shows trends over time.

Example

```
fig = px.line(df, x='Year', y='Sales')
```

4.3.3 Bar Chart

Purpose

Compares categories.

Example

```
fig = px.bar(df, x='Country', y='Population')
```

4.3.4 Bubble Chart

Purpose

Scatter plot with variable bubble size.

Example

```
fig = px.scatter(  
    df,  
    x='GDP',  
    y='LifeExpectancy',  
    size='Population'  
)
```

4.3.5 Histogram

Purpose

Shows frequency distribution.

Example

```
fig = px.histogram(df, x='Age')
```

4.3.6 Pie Chart

Purpose

Shows proportions.

Example

```
fig = px.pie(  
    df,  
    values='Sales',  
    names='Category'  
)
```

4.3.7 Sunburst Chart

Purpose

Visualizes hierarchical data.

Example

```
fig = px.sunburst(  
    df,  
    path=['Country', 'State', 'City'],  
    values='Sales'  
)
```

4.4 AIRLINE DATASET OVERVIEW

The airline reporting dataset contains:

- Flight dates
- Departure airports
- Arrival airports
- Delay information
- Flight performance metrics

Used for:

- Dashboard creation
- Interactive analytics
- Trend visualization

PLOTLY GRAPH OBJECTS VS PLOTLY EXPRESS

Feature	Graph Objects	Plotly Express
Level	Low-level	High-level
Flexibility	Very high	Moderate
Code Length	More	Less
Ease of Use	Complex	Easy
Best For	Advanced customization	Quick visualization

IMPORTANT PLOTLY FUNCTIONS

Plot Type	Plotly Express Function
Scatter Plot	<code>px.scatter()</code>
Line Plot	<code>px.line()</code>
Bar Chart	<code>px.bar()</code>
Histogram	<code>px.histogram()</code>
Pie Chart	<code>px.pie()</code>
Bubble Chart	<code>px.scatter(size=)</code>
Sunburst	<code>px.sunburst()</code>

KEY TAKEAWAYS

Dashboards

- Provide interactive business insights
- Improve decision-making
- Support real-time analytics

Plotly

- Interactive visualization library
- Supports 40+ chart types
- Used in dashboards and analytics apps

Plotly Graph Objects

- Low-level detailed customization

Plotly Express

- High-level simplified plotting

Interactive Features

Plotly charts support:

- Hover effects
- Zooming
- Panning
- Filtering
- Drill-down analysis

[Plotly Basics: Scatter, Line, Bar, Bubble, Histogram, Pie, Sunburst](#)

4.5 INTRODUCTION TO DASH

4.5.1 What is Dash?

Dash is a Python framework used for creating:

- Interactive dashboards
- Web applications
- Real-time visual analytics

Built on:

- Flask
- React.js
- Plotly.js

4.5.2 Components of Dash

A. Core Components (dcc)

Interactive elements like:

- Graphs
- Input boxes
- Sliders
- Dropdowns

Import

```
from dash import dcc
```

B. HTML Components

HTML elements in Python.

Import

```
from dash import html
```

Examples

```
html.H1()  
html.Div()  
html.P()
```

4.5.3 Basic Dash App Structure

```
from dash import Dash, html  
app = Dash(__name__)  
app.layout = html.Div([  
    html.H1("Hello Dash")  
)  
if __name__ == '__main__':  
    app.run_server(debug=True)
```

4.5.4 Dash Graph Component

```
dcc.Graph(figure=fig)
```

Example:

```
html.Div([  
    dcc.Graph(figure=fig)  
)
```

4.5.5 Dash Input Component

```
dcc.Input(  
    value='2010',  
    type='number')
```

4.5.6 Dash Dropdown

```
dcc.Dropdown(  
    options=[  
        {'label':'Option 1','value':'1'}],  
    value='1'  
)
```

[Dash Basics: HTML and Core Components](#)

4.6 DASH CALLBACK FUNCTION

4.6.1 What is Callback?

A callback automatically updates outputs whenever input values change.

Syntax

```
@app.callback(  
    Output(component_id, property),  
    Input(component_id, property)  
)
```

4.6.2 Callback Example

```
from dash import Dash, dcc, html  
from dash.dependencies import Input, Output  
app = Dash(__name__)  
app.layout = html.Div([  
    dcc.Input(id='input-year', value='2010'),  
    html.Div(id='output')  
)  
@app.callback(  
    Output('output', 'children'),  
    Input('input-year', 'value')  
)  
def update_output(year):  
    return f'Entered Year: {year}'  
if __name__ == '__main__':  
    app.run_server(debug=True)
```

[Add Interactivity: User Input and Callbacks](#)

4.7 FLIGHT DELAY DASHBOARD LAB WORKFLOW

Steps

1. Install packages
2. Create flight_delay.py
3. Load airline dataset
4. Create Dash layout
5. Add graphs
6. Create callback functions
7. Run server
8. Launch application

[Flight Delay Time Statistics Dashboard](#)

IMPORTANT DASH CONCEPTS

Concept	Meaning
Layout	Structure of dashboard
Callback	Makes app interactive
Input	User interaction component
Output	Updated component
Graph	Plotly chart inside Dash

MODULE 5: FINAL PROJECT IMPORTANT POINTS

5.1 PRACTICE PROJECT (WILDFIRE AUSTRALIA)

Part 1 Visualization Tasks

You must know:

- Line chart
- Bar plot
- Pie chart
- Histogram
- Scatter plot
- Folium maps

Important Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import folium
```

Dashboard Tasks

User selects:

- Region
- Year

Display:

1. Pie chart
2. Bar chart

[Practice Assignment: Part 1 - Analyzing wildfire activities in Australia](#)

[Practice Assignment: Part 2 - Creating Dashboards](#)

5.2 FINAL PROJECT (AUTOMOBILE SALES)

Recession Years

- 1980
- 1981–82
- 1991
- 2000–01
- 2007–09
- 2020

Important Visualizations

Task	Plot Type
Sales trend	Line chart
Ad expenditure vs sales	Scatter/Line
Vehicle comparison	Seaborn plot
GDP comparison	Subplots
Seasonality impact	Bubble plot
Price vs sales	Scatter
Ad expenditure share	Pie chart
Unemployment effect	Line plot

Dashboard Requirements

Dropdowns

- Statistics Type
- Year

Two Report Types

1. Recession Statistics
2. Yearly Statistics

[Final Assignment: Part 1 - Create Visualizations using Matplotlib, Seaborn & Folium](#)

[Final Assignment: Part 2 - Create Dashboard with Plotly and Dash](#)

5.3 IMPORTANT CALLBACK STRUCTURE

```
@app.callback(  
    Output('output-container', 'children'),  
    [  
        Input('dropdown-statistics', 'value'),  
        Input('select-year', 'value')  
    ]  
)
```

Common Errors

1. Invalid Syntax

Usually caused by:

- Missing brackets
- Missing commas

2. Callback Errors

- Wrong component IDs
- Incorrect input/output order

3. Graph Not Displaying

Use:

```
elif selected_statistics == 'Yearly Statistics':
```

ONE-LINE DEFINITIONS

Folium

Python library for interactive geospatial visualization.

Choropleth Map

Map with shaded regions based on data values.

Plotly

Interactive visualization library.

Dash

Framework for building web dashboards.

Callback

Function automatically triggered when input changes.

Seaborn

Statistical visualization library built on Matplotlib.

Waffle Chart

Grid-based visualization for categorical proportions.

Word Cloud

Visualization of word frequency in text.
